

BPM 설계

1. Full Instance VS Job

1) Full Instance 방식(전통적)

```
[Instance 1]
Thread 1 → start → task → gateway → task → wait (persist) → ...
                                         ↑ 여기서 DB 저장, 메모리 해제
```

- 프로세스 인스턴스 시작 시 실행 컨텍스트 로드
- 연속 실행: 가능한한 다음 단계까지 이어서 처리
- wait state 도달 시 DB 에 persist 후 메모리 해제

(1) 핵심특징

- 한 번 시작하면 가능한한 끊지 않고 연속 실행
- 실행 컨텍스트가 상대적으로 길게 유지됨
- 같은 트랜잭션 내에서 여러 노드 실행 가능

(2) 장점

- 구현이 단순함
- DB round trip 횟수 적음
- 짧은 프로세스에선 매우 빠름

(3) 단점

- 확장성 제한: 인스턴스 수가 스레드/메모리와 밀접
- 장애시 복구 복잡
- 장기 대기 프로세스에 비효율

2) Job 중심 실행

[Job Queue]	[Timeline]
Job(inst1, node1) → Worker A	t=0
Job(inst1, node2) → Worker B	t=1 (A 완료 후)
Job(inst1, node3) → Worker A	t=2 (B 완료 후)

△ 중요: 같은 instance 는 동시에 여러 워커가 처리하지 않음.
instance-level lock/lease 로 보호되며, "다른 워커 처리"는 시간적으로 분리된 실행임.
즉, 하나의 Process Instance 내부 상태 변경은 항상 순차적으로 처리되며, 이는 Instance 상태의 일관성을 보장하기 위한 설계이다.

- 실행을 작은 단위 Job 으로 분리
- DB 가 Single Source of Truth
- 워커는 FOR UPDATE SKIP LOCKED 로 Job 획득
- 한 Job 끝나면 다음 단계는 "새 Job"으로 생성

(1) 핵심 특징

- 흐름을 단계별로 끊음
- 실행 단위는 항상 Job
- 실행 상태는 항상 DB 에 기록

(2) 장점

- 수평 확장 용이(워커만 추가)
- 장애 자동 복구
- 장기 대기 프로세스에 최적
- Timer/Retry/Backoff 구조적으로 쉬움

(3) 단점

- Lock/Lease 설계 필요
- DB I/O 증가

3) 비교 요약

항목	Full Instance	Job Model
실행 단위	연속 흐름	단계별 Job
확장성	수직	수평
장애 복구	복잡	자연스러움
DB 의존도	낮음	높음
장기 프로세스	불리	유리
Timer/Retry	구현 복잡	구조적으로 쉬움

워크플로우는 장기 실행, 외부 시스템 연동, 재시도, 타이머와 같은 특성이 빈번하게 발생한다. 이러한 특성을 고려할 때 실행 상태를 메모리에 유지하는 방식보다 데이터베이스를 Source of Truth 로 사용하는 Job 기반 실행 모델이 확장성과 안정성 측면에서 유리하다.

<추가>

PXM 은 실행을 Job 단위로 분해할 뿐 아니라, 재시도(Retry), 타이머 대기(Timer), 사용자 승인 후 재개(Resume), 리마인더(Reminder), 에스컬레이션(Escalation)과 같은 예약성/재개성 동작도 공통적으로 engine_jobs 기반으로 처리한다.

즉, 런타임에서 “나중에 다시 해야 하는 일”은 별도의 retry/timer/reminder 전용 테이블로 흩어지기보다, 공통 Job Queue 모델 안에서 `type` 과 `run_at` 으로 표현하는 것을 원칙으로 한다.

2. Node 모델

1) 범용 노드 + 스크립트 기반 설계 (회의때 나온 방식)

일부 초기 워크플로우 시스템이나 사내 구현에서는 범용 노드에 스크립트를 작성하여 모든 로직을 처리하는 방식이 사용되었다.

이 방식에서는 노드 내부에 스크립트를 작성하여 모든 동작을 구현한다.

```
Node
└─ script
    ├── 외부 API 호출
    ├── 데이터 가공
    ├── 조건 판단
    └─ 승인 여부 처리
```

즉 하나의 노드 안에서 위의 로직이 모두 구현된다.

노드의 의미는 노드 타입이 아니라 **스크립트 내용에 의해 결정**된다.

그러나 BPMN 기반 BPM 엔진들은 일반적으로 노드 타입 기반 모델을 사용한다.

=> BPMN 기반 엔진에서는 노드의 역할이 명확하게 구분되며, Service Task, User Task, Gateway 등 타입별로 실행 의미가 정의된다.

2) 범용노드 방식의 문제점

(1) 프로세스 구조의 불명확성

노드의 역할이 스크립트에 의해 결정되기 때문에 프로세스 다이어그램만 보고는 실제 동작을 이해하기 어렵다.

```
ex)
[Node]
└─ script: 200 lines
```

해당 노드가 서비스 호출인지, 승인 처리인지 등을 다이어그램만으로는 알 수가 없다.

(2) 예측 가능한 실행이 어려움

엔진이 노드의 동작을 사전에 알 수 없다. 엔진 입장에서는 이것이 외부 호출인지, 승인인지 등을 실행 전에 알 수가 없어서 추적, 재시도 정책, 타이머 처리등의 어려움이 있다.

(3) 디버깅 및 운영이 어렵다

문제가 발생했을때 해당 경로로 간 이유를 찾으려면 스크립트를 직접 분석해야한다.
실행의 흐름이 다이어그램이 아니라 코드 안에 숨게 된다. (1 번과 비슷)

3) 타입 기반 노드 설계

타입 기반 노드 설계는 각 노드가 명확한 역할을 가진다.

```
Start
Service
Gateway
Approval
Timer
End

Start
↓
Service (HR API 조회)
↓
Gateway (조건 분기)
↓
Approval (팀장 승인)
↓
Service (권한 시스템 호출)
↓
End
```

노드의 동작은 노드 타입에 의해 결정된다.

4) 타입 기반 노드의 장점

범용 노드의 단점이 타입 기반 노드의 장점이다.

(1) 프로세스 구조가 명확하다.

노드 타입만 보아도 프로세스의 의미를 이해할 수 있다.

```
Service → Gateway → Approval
외부 시스템 호출 -> 조건 분기 -> 사용자 승인
```

즉, 다이어그램만으로 프로세스 구조 이해가 가능.

(2) 엔진이 동작 예측 가능

노드 타입이 정의되어 있기 때문에 엔진은 각 노드의 동작을 명확하게 알 수 있다.

Node Type	엔진 동작
Service	외부 API 호출
Gateway	조건 평가
Approval	사용자 Task 생성
Timer	지연 실행

이를 통해 Retry 정책, Timer 스케줄링, Task 생성, 실행 로그 추적 구현이 용이해 진다.

(3) 실행 흐름 추적과 운영의 용이함

실행 로그는 다음과 같이 노드 단위로 기록될 수 있다.(변경 가능 중요한건 타입별 로그)

```
NODE_STARTED: Service
NODE_COMPLETED: Service
NODE_STARTED: Gateway
NODE_COMPLETED: Gateway
NODE_STARTED: Approval
```

이를 통해 실행 흐름을 시각적으로 추적할 수 있다.

5) PXM 에서 지원 계획인 Node 타입

(1) 지원 Node 타입

pxm 은 BPMN 전체 요소를 구현하지 않고 실제 업무 워크플로우에 필요한 핵심 노드만 지원할 예정.
현재 계획한 노드는 다음과 같다.

TYPE	Start	Service	Gateway	Approval	Timer	End
설명	프로세스 시작	외부 시스템 호출	조건 분기	사용자 승인	지연 실행	프로세스 종료

따라서 PXM 의 프로세스 정의는

타입 기반 노드 + 연결 흐름 선(Edge) 구조를 기반으로 설계한다.

노드는 실행 단위이며, 노드 간의 실행 흐름은 Edge 로 연결된다.

Start 성격의 Node 는 사용자 입력(Form), 수동 실행, Timer, Webhook 등 다양한 방식으로 프로세스 진입점을 형성할 수 있다.

(2) Node 타입 개요

• Start Node

Start Node 는 프로세스 실행의 진입 지점이다.

Process Instance 생성 이후,
실행 토큰(Token)은 Start Node 에서 생성되어
Edge 를 따라 다음 Node 로 이동한다.

Start Node 는 다양한 Trigger 방식으로 실행될 수 있다.

대표적으로 다음 방식이 사용될 수 있다.

- Form Start : 사용자 입력 폼 기반 실행
- Manual Start : 사용자의 수동 실행
- Webhook Start : 외부 HTTP 요청 기반 실행
- Timer Start : 일정 기반 자동 실행

• Service Node

Service Node 는 외부 시스템 호출 또는 자동화된 로직을 수행하는 노드이다.

대표적으로 HTTP API 호출, 내부 서비스 실행, 데이터 조회 등의 작업을 수행한다.
Service Node 는 성공/실패 결과에 따라 다음 실행 경로를 결정할 수 있다.

- **Gateway Node**

Gateway Node 는 조건 평가를 통해 프로세스 실행 경로를 분기하는 노드이다.

Execution Context 의 데이터를 기반으로 조건식을 평가하여 하나 이상의 Edge 로 실행 흐름을 전달한다.

PXM 에서는 XOR, AND, OR 이 사용된다.

- **Approval Node**

Approval Node 는 사용자 참여가 필요한 작업을 처리하는 노드이다.

이 노드는 실행 중단 상태(Waiting)가 되며 사용자의 승인 또는 반려 입력이 발생하면 프로세스 실행이 재개된다.

Approval Node 는 Task 생성과 연계되어 동작한다.

- **Timer Node**

Timer Node 는 지정된 시간 이후에 프로세스 실행을 재개하기 위한 노드이다.

지연 실행, 일정 시간 대기, SLA 처리 등의 시간 기반 제어를 위해 사용된다.

Timer Node 는 내부적으로 예약된 Job 을 생성하여 지정된 시점에 다음 노드를 실행한다.

- **End Node**

End Node 는 프로세스 실행이 종료되는 지점이다.

실행 중인 Token 이 End Node 에 도달하면 해당 실행 흐름은 완료 상태가 된다.

모든 실행 Token 이 종료되면,
Process Instance 는 Completed 상태가 된다.

3. Edge 모델

1) Node 간 실행 흐름

앞서 정의한 Node 는 프로세스의 실행 단위이다.

프로세스는 여러 Node 가 연결되어 하나의 실행 흐름을 구성한다.

따라서 Node 간의 실행 순서를 정의하기 위해 Node 사이의 연결 구조가 필요하다.

이 연결 구조를 Edge 로 정의한다.

```
Node — Edge —> Node
```

Edge 는 Node 간의 실행 흐름(Flow)을 나타낸다.

2) Edge 의 역할

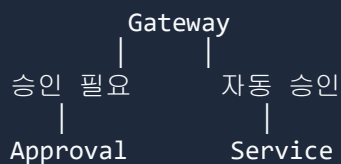
Edge 는 다음 역할을 가진다.

1. Node 간 실행 순서를 정의한다.
2. 분기 조건을 표현한다.
3. 다음 실행 node 를 결정한다.

즉, Edge 는 단순 연결선이 아니라 프로세스 실행 경로를 정의하는 요소이다.

3) 분기 처리와 Edge

워크플로우에서는 특정 조건에 따라 다음 실행 경로가 달라질 수 있다.



이때 분기는 Gateway Node 에서 발생하지만, 각 경로의 선택 조건은 Edge 에 정의된다.

```
Gateway
├─ Edge (condition: amount > 1000) → Approval
└─ Edge (condition: amount <= 1000) → Service
```

즉, Gateway Node 는 분기의 의미를 가지고, Edge 는 어떤 조건에서 어떤 경로로 이동할지 정의한다.

4) Edge 에 조건을 두는 이유

(1) 조건은 경로의 속성

분기 조건은 Gateway 자체의 상태라기보다 특정 경로로 이동하기 위한 규칙이다.

(2) 다중 분기 표현이 단순

Edge 조건 모델에서는 여러 경로를 자연스럽게 표현할 수 있다.

```
Gateway
├─ Edge(cond: status == "OK")
├─ Edge(cond: status == "FAIL")
├─ Edge(cond: status == "RETRY")
└─ Edge(default)
```

각 경로가 독립적인 조건을 가지므로 복잡한 분기구조도 단순하게 표현할 수 있다.

(3) BPMN 모델과 동일한 구조

BPMN 2.0에서는 Node 간 연결을 Sequence Flow 라고 하며 분기 조건은 Sequence Flow 에 정의된다.

5) Edge 구조

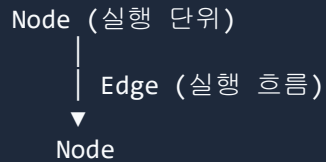
PXM 의 Edge 는 다음 정보를 가진다.

필드	설명
source	시작 Node
target	다음 Node
condition	분기조건
isDefault	기본 경로 여부

Edge 는 Node 간의 실행 흐름과 분기 조건을 함께 정의하는 핵심 요소이다.

6) 결론

pxm의 프로세스 모델은 다음 구조를 따른다.



여기서

- Node는 실행 동작을 정의하고
- Edge는 실행 경로를 정의한다.

특히 분기 조건은 **Edge의 속성으로 표현**하여 엔진 실행 모델을 단순하게 유지하면서 유연한 분기 구조를 지원하도록 설계한다.

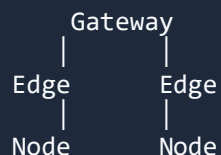
4. Gateway 모델

1) Gateway Node

Gateway Node는 워크플로우 실행 중 **분기 또는 합류(join)**를 처리하는 노드이다.

Gateway는 단순 연결 노드가 아니라

실행 경로를 제어하는 제어 흐름 노드(Control Flow Node)이다.



Gateway는 여러 Edge를 통해 다음 실행 경로를 결정한다.

2) Gateway Type

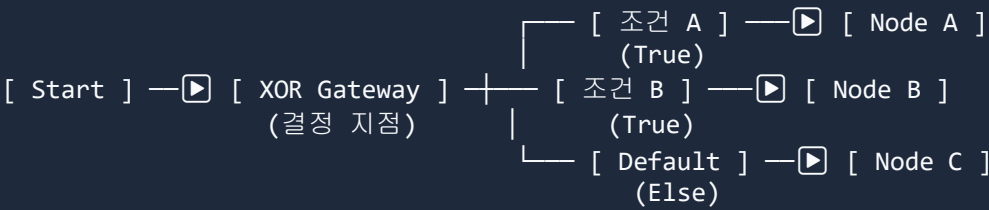
Gateway 는 분기 방식에 따라 여러 유형이 존재한다.

PXM 은 BPMN 모델을 참고하여 다음 Gateway 타입을 지원한다.

Gateway Type	의미
Exclusive (XOR)	조건 중 하나의 경로만 선택
Parallel (AND)	모든 경로를 동시에 실행
Inclusive (OR)	조건에 맞는 여러 경로 실행

3) Exclusive Gateway (XOR)

Exclusive Gateway 는 여러 조건 중 **하나의 경로만 선택하여 실행**한다.

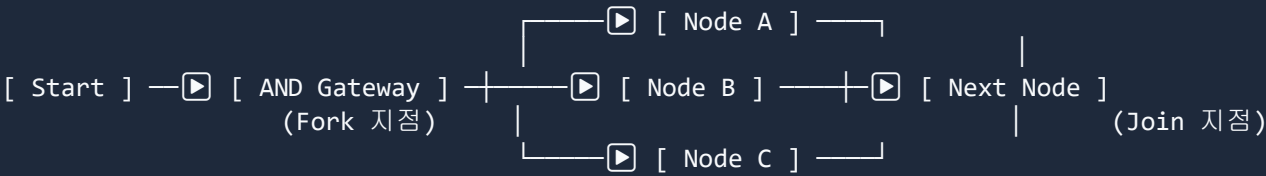


엔진 동작

조건을 순서대로 평가 첫 번째 true 조건 Edge 선택

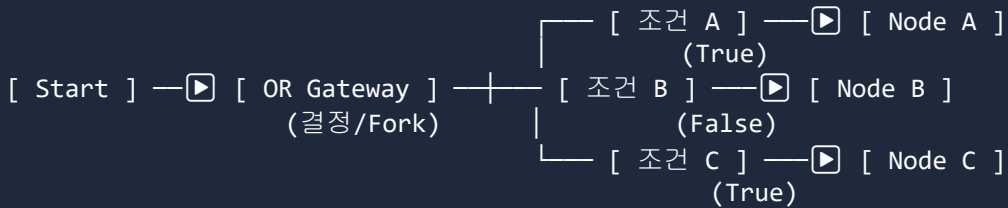
4) Parallel Gateway (And)

Parallel Gateway 는 **모든 경로를 동시에 실행**한다.



5) Inclusive Gateway (OR)

Inclusive Gateway 는 조건을 평가하여 조건에 맞는 여러 경로를 동시에 실행할 수 있다.

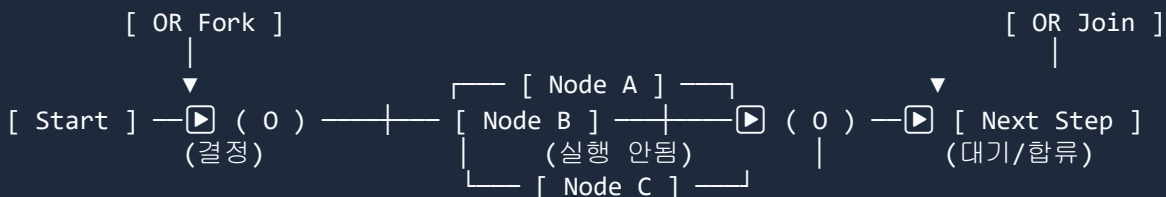


* Node A 와 Node C 가 병렬로 실행

즉, XOR 과 AND 의 중간 형태이다.

6) Gateway Join

Gateway 는 분기뿐 아니라 여러 실행 흐름을 합류(join)하는 역할도 한다.



* 예: 조건에 의해 A, C 만 실행되었다면, Join 게이트웨이는 B 를 기다리지 않고 A, C 가 모두 도착하는 순간 즉시 다음 단계로 진행.

Inclusive Gateway Join 에서는
실제로 실행된 경로만 기다린 후 다음 단계로 진행한다.

즉, 엔진은 다음 정보를 알아야 한다.

현재 Gateway 에서 실제로 몇 개의 경로가 실행되었는가?

이를 위해 PXM 은 **Token 기반 실행 모델**을 사용한다.

5. Process Instance 모델

1) 개요

Process Instance 는 프로세스 정의의 **실행 단위**이다.

사용자가 프로세스를 실행하면 프로세스 정의로부터 새로운 Process Instance 가 생성되며 이 Instance 는 워크플로우 실행 상태와 데이터를 관리하는 **Runtime Root Entity** 역할을 한다.

즉, Process Instance 는 다음과 같은 실행 정보를 포함한다.

- 실행 상태
- 실행 Context
- 실행 Token
- 생성된 Task
- 실행 Log
- 예약된 Job

모든 Runtime 데이터는 **instance_id** 를 기준으로 연결된다.

2) Instance 생성

Process Instance 는 다음과 같은 흐름으로 생성된다.

```
Process Definition
  ↓
Instance 생성
  ↓
Start Node 실행
  ↓
Workflow 실행 시작
```

```
process_definition : "권한 요청 프로세스"
```

```
instance 생성
```

```
instance_id : inst_1001
```

```
requester : 김민수
```

Instance 가 생성되면 엔진은 **START Job** 을 생성하여 실행을 시작한다.

Process Instance 는 사용자에 의한 수동 실행뿐 아니라 외부 시스템의 HTTP 요청(Webhook)을 통해서도 생성될 수 있다.

이 경우 Node API 는 Webhook 요청을 수신한 뒤 대상 Process Definition 을 식별하고 새로운 Process Instance 를 생성한 후 START Job 을 등록하여 워크플로우 실행을 시작한다.

```
외부 시스템 HTTP 요청
```

```
↓
```

```
Webhook Endpoint 수신
```

```
↓
```

```
Process Definition 식별
```

```
↓
```

```
Process Instance 생성
```

```
↓
```

```
START Job 생성
```

```
↓
```

```
Workflow 실행 시작
```

3) Intance 구성 요소

Process Instance 는 다음 Runtime 요소들을 포함한다.

구성 요소	설명
context	실행 데이터 저장
tokens	현재 실행 위치
tasks	사용자 작업
jobs	실행 작업
execution_logs	실행 이력

```
Process Instance
├── context
├── tokens
├── tasks
├── jobs
└── execution_logs
```

이 구조를 통해 pxm 엔진은 프로세스 실행 상태를 일관되게 관리할 수 있다.

4) Instance 상태

Process Instance 는 실행 과정에서 다음 상태를 가진다.

상태	설명
CREATED	Instance 생성
RUNNING	실행 중
WAITING	사용자 입력 또는 외부 이벤트 대
COMPLETED	정상 종료
FAILED	실행 실패
TERMINATED	운영자에 의해 종료

예)

```
CREATED
↓
RUNNING
↓
WAITING
↓
RUNNING
↓
COMPLETED
```

또는 오류 발생 시

```
RUNNING
↓
FAILED
```

운영자가 강제 종료할 경우

```
RUNNING
↓
TERMINATED
```

5) Instance 와 Token

PXM 은 Token 기반 실행 모델을 사용한다.

Token 은 워크플로우의 **현재 실행 위치**를 나타내는 객체이며,
Instance 는 이 Token 들을 관리한다.

```
Instance
├ token_1 → Node A
└ token_2 → Node B
```

Parallel Gateway 드루이 구조에서는 하나의 Instance 에 여러 Token 이 동시에 존재할 수 있다.

6) Instance 와 Job Model

PXM 엔진은 Job 기반 실행 모델을 사용한다.
따라서 Node 실행은 다음 흐름으로 이루어진다.

```
Instance 생성
  ↓
Token 생성
  ↓
Job 생성
  ↓
Worker 실행
  ↓
Token 이동
```

즉, Instance 는 실행 컨텍스트를 보관하고 Job 은 실제 수행하는 역할을 한다.

7) 정리

Process Instance 모델을 통해 pxm 엔진은 다음 특성을 제공한다.

- 프로세스 실행 상태의 일관된 관리
- Token 기반 실행 지원
- Job 기반 분산 실행 지원
- 실행 데이터 및 로그의 통합 관리

Process Instance 는 PXM Runtime 구조에서 **모든 실행 데이터를 연결하는 중심 엔티티** 역할을 한다.

참고) Process Instance 개념은 대부분의 Workflow 엔진에서 공통적으로 사용. Camunda, Temporal 등의 시스템에서도 프로세스 정의와 실행 인스턴스를 분리하여 워크플로우 실행 상태를 관리

6. Token 기반 실행 모델

1) Token 개념

PXM 의 워크플로우 실행은 **Token 이동 모델**을 기반으로 한다.

Token 은 프로세스 실행 흐름을 나타내는 단위이다.

```
Node 실행
↓
Token 이동
↓
Edge 따라 다음 Node
```

즉 프로세스 실행은 **Token** 이 그래프를 따라 이동하는 과정이다.

2) Token 이동

프로세스 실행 중 Token 은 다음 과정을 반복한다.

```
Token → Node 도착
Node 실행
Edge 선택
Token 이동
```

Gateway 에서 Token 은 다음과 같이 분기된다.

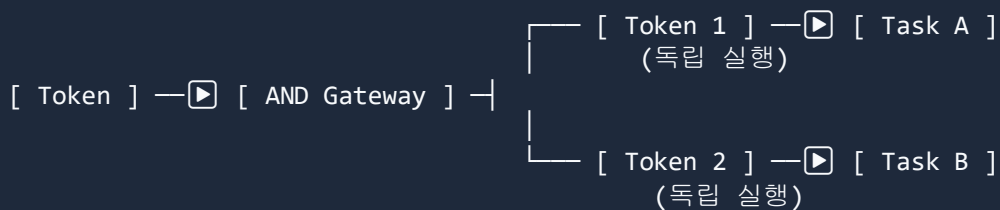
XOR → Token 1 개 유지

AND → Token 여러 개 생성

OR → 조건에 따라 여러 Token 생성

3) Parallel 실행

Parallel Gateway 에서는 각 Edge 마다 새로운 Token 이 생성된다.



각 Token 은 독립적으로 실행된다.

4) Inclusive Join

Inclusive Gateway Join 에서는 다음 조건이 만족될 때까지 대기한다.

현재 Gateway 에서
실제로 실행된 Token 이 모두 도착

즉

A 실행
B 실행

이면

A Token
B Token

두 Token 모두 도착해야 다음 Node 로 진행한다.

5) Job 모델과 Token

PXM 은 Job 기반 실행 모델을 사용한다.
따라서 Token 이동은 다음과 같이 처리된다.

```
Token 도착
  ↓
Node 실행 Job 생성
  ↓
Node 완료
  ↓
다음 Token 생성
  ↓
다음 Job 생성
```

즉 Token 이동이 곧 Job 생성의 기준이 된다.

6) 실행 흐름 요약

PXM 의 실행 흐름은 다음 구조로 동작한다.

```
Node 실행
↓
Token 생성 또는 이동
↓
Edge 선택
↓
다음 Node 실행
```

이 모델은

- Parallel 실행
- Inclusive Gateway
- Retry / Timer

등을 자연스럽게 지원한다.

7. Execution Context 모델

1) 개요

Execution Context 는 프로세스 실행 중 사용되는 데이터를 저장하는 영역이다.
PXM 에서는 Execution Context 를 Process Instance 단위로 관리한다.

즉, 하나의 Process Instance 에는 하나의 Context 가 존재하며,
프로세스 실행 과정에서 모든 Node 는 동일한 Context 를 읽고 수정할 수 있다.

Execution Context 는 다음과 같은 용도로 사용된다.

- Start/Form Node 입력 데이터 저장
- Service Node 실행 결과 저장
- Gateway 조건 평가
- Approval Node 결과 기록
- 프로세스 실행 결과 저장

이 Context 는 프로세스 인스턴스가 생성될 때 초기화되며, 프로세스 실행이 진행되는 동안 지속적으로 갱신된다.

2) Instance-Level Context

PXM 의 Execution Context 는 **Process Instance** 단위의 공유 데이터 구조이다.
즉, 하나의 Instance 안에서 실행되는 모든 Node 는 동일한 Context 를 사용한다.

```
Start/Form Node
  ↓
Service Node
  ↓
Gateway
  ↓
Approval Node
  ↓
End
```

이 모든 Node 는 동일한 Context 를 읽고 수정한다.

3) Context 초기화

Execution Context 는 Start/Form Node 에서 초기 입력 데이터로 초기화된다.

이 초기 데이터는 사용자 Form 입력일 수도 있고, 외부 시스템이 Webhook 으로 전달한 HTTP Request Payload 일 수도 있다.

Form Start

→ 사용자 입력값이 `ctx.input` 으로 저장

Webhook Start

→ HTTP request body 가 `ctx.input` 또는 `ctx.vars` 초기값으로 저장

ex) 사용자가 다음 정보를 입력한 경우

사용자: 1001

시스템: HR

권한: READ

```
{
  "input": {
    "userId": 1001,
    "system": "HR",
    "permission": "READ"
  },
  "vars": {},
  "tasks": {},
  "result": {}
}
```

이후 각 Node 의 실행 결과가 Context 에 누적된다.

4) Context 권장 구조

Execution Context 내부 데이터는 프로세스마다 자유롭게 정의될 수 있다.

다만, 운영 및 가독성을 위해 다음과 같은 구조로 가고자 한다.

영역	용도
input	Start/Form 입력 데이터
vars	Service Node 결과 및 실행 중 생성 데이터
tasks	Approval / User Task 결과
result	프로세스 최종 결과

이 구조를 사용하면 다음과 같은 장점이 있다.

- 입력 데이터와 실행 데이터를 명확히 구분
- 디버깅 및 실행 추적 용이
- 병렬 실행시 데이터 충돌 가능성 감소
- Runtime Trace 가독성 향상

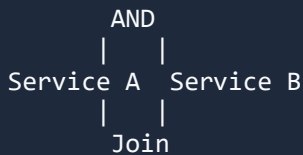
5) 병렬 실행과 Context

PXM 은 AND / OR Gateway 를 통해 병렬 실행을 지원한다.

병렬 실행시 여러 Branch 가 동시에 실행될 수 있으며,
이 경우 동일한 Context 경로를 동시에 수정하면 충돌이 발생할 수 있다.

이를 방지하기 위해 다음 패턴을 권장한다.

- 병렬 Branch 는 vars 또는 tasks 영역에 각각의 결과를 기록
- 병렬 결과는 Join 이후 Node 에서 result 영역으로 병합



```
#Service A
ctx.vars.serviceA = ...
#Service B
ctx.vars.serviceB = ...

Join 이후 Node
ctx.result.final = combine(serviceA, serviceB)
```

이 패턴을 사용하면 병렬 실행 시 Context 충돌을 최소화할 수 있다.

(이 부분은 위처럼 안하고 그냥 lost update 현상을 그대로 가지고 가는게 나을지
고민중..combine 정의가 고민되서..)

6) 다른 Workflow 엔진과의 비교

Execution Context 와 유사한 개념은 대부분의 Workflow / BPM 엔진에서 사용된다.

- Camunda → **Process Variables**
- Temporal → **Workflow State**
- Zeebe → **Variables**

이들 엔진 역시 프로세스 실행 중 데이터를 Instance 단위의 공유 상태로 관리한다.

즉, Execution Context 를 Instance 단위로 관리하는 구조는 Workflow 엔진에서 널리 사용되는 일반적인 설계 방식이다.

7) Execution Context 특징

Execution Context 는 프로세스 실행 중 사용되는 데이터를 저장하는 영역이며, PXM 에서는 이를 **Process Instance 단위의 공유 Context** 로 관리한다.

Execution Context 의 주요 특징은 다음과 같다.

- Camunda → **Process Variables**
- Temporal → **Workflow State**
- Zeebe → **Variables**

이들 엔진 역시 프로세스 실행 중 데이터를 **Instance 단위의 공유 상태**로 관리한다.

- Instance 단위의 공유 데이터
- Start/Form Node 입력으로 초기화
- Node 실행 결과가 누적되는 구조
- Gateway 조건 평가 및 실행 로직에 사용
- 병렬 실행시에도 동일 Context 를 기반으로 동작

이를 통해 PXM 은 프로세스 실행 중 발생하는 데이터를 일관된 방식으로 관리할 수 있다.

8. Execution Log / Event 모델

1) 개요

pxm 엔진은 프로세스 실행과정에서 발생하는 실행 이력을 기록한다.
이 실행 이력은 다음 두 가지 목적을 가진다.

1. 엔진 내부 실행 추적 및 운영 관리
2. 외부 시스템 또는 UI 에 전달할 이벤트 생성

이를 위해 PXM 은 실행 이력을 다음 두 계층으로 관리한다.

구분	목적
Execution Log	엔진 내부 실행 기록
Event Outbox	외부 시스템 전달 이벤트

이 두 구조는 목적과 사용 대상이 다르다.

2) Execution Log

Execution Log 는 프로세스 실행 과정에서 발생하는 **엔진 내부 실행 기록**이다.
Execution Log 는 다음과 같은 목적으로 사용된다.

- Node 실행 흐름 추적
- 오류 분석 및 디버깅
- 실행 타임라인 제공
- 운영 모니터링

Execution Log 는 일반적으로 다음과 같은 이벤트를 기록한다.

Event	설명
NODE_STARTED	Node 실행 시작
NODE_COMPLETED	Node 실행 완료
NODE_FAILED	Node 실행 실패
TASK_CREATED	사용자 작업 생성
TASK_COMPLETED	사용자 작업 완료

```
{
  "log_type": "NODE_COMPLETED",
  "instance_id": "inst_1",
  "token_id": "tok_1",
  "node_id": "service_hr_lookup",
  "status": "COMPLETED",
  "timestamp": "2026-03-06T16:01:10Z"
}
```

Execution Log 는 주로 **운영자 UI** 에서 실행 타임라인을 구성하거나 문제 분석에 사용된다.

3) Event Outbox

Event Outbox 는 프로세스 실행 중 발생한 주요 이벤트를 외부 시스템에 전달하기 위한 이벤트 저장소이다.

Execution Log 가 엔진 내부 기록이라면,

Event Outbox 는 **외부 소비자가 구독할 수 있는 이벤트 스트림** 역할을 한다.

Event Outbox 는 다음과 같은 용도로 사용될 수 있다.

- UI 상태 갱신
- 알림 전송
- Webhook 호출
- 외부 시스템 연동
- 이벤트 기반 분석

예)

Event	설명
INSTANCE_CREATED	인스턴스 생성
INSTANCE_RUNNING	실행 시작
INSTANCE_WAITING	대기 상태
INSTANCE_COMPLETED	실행 완료
INSTANCE_FAILED	실행 실패
NODE_STARTED	노드 실행 시작
NODE_COMPLETED	노드 실행 완료
TASK_CREATED	사용자 작업 생성
TASK_COMPLETED	사용자 작업 완료
RETRY_SCHEDULED	재시도 예약

```
{
  "event_type": "NODE_COMPLETED",
  "instance_id": "inst_1",
  "token_id": "tok_1",
  "node_id": "service_hr_lookup",
  "occurred_at": "2026-03-06T16:01:10Z",
  "payload": {
    "from_node_id": "start_1",
    "to_node_id": "service_hr_lookup",
    "status": "COMPLETED"
  }
}
```

참고로 Webhook 은 두 가지 방향으로 사용될 수 있다.

하나는 외부 시스템의 HTTP 요청을 받아 Process Instance 를 시작하는 **Inbound Webhook Trigger** 이고,
 다른 하나는 Event Outbox 를 기반으로 외부 시스템에 이벤트를 전달하는 **Outbound Webhook Call** 이다.

4) Execution Log 와 Event Outbox 의 차이

Execution Log 와 Event Outbox 는 역할이 다르다.

구분	Execution Log	Event Outbox
목적	내부 실행 기록	외부 이벤트 전달
사용 대상	운영자 / 관리자	UI / 외부 시스템
데이터 성격	상세 실행 로그	상태 변화 이벤트
저장 범위	모든 실행 이력	주요 이벤트

즉,

- **Execution Log** 는 엔진 내부 운영 기록
 - **Event Outbox** 는 외부 소비자를 위한 이벤트
- 역할을 가진다.

5) DB Source of Truth 와의 관계

PXM 의 실행 상태는 데이터베이스를 **Source of Truth** 로 관리한다.

즉, 실제 실행 상태는 다음 데이터에 저장된다.

- process_instance
- token
- task

Execution Log 와 Event Outbox 는 이 **상태 변화**를 기록하는 보조 구조이다.

데이터	역할
process_instance	현재 실행 상태
token	실행 위치
execution_log	내부 실행 기록
event_outbox	외부 이벤트 전달

Event Out box 는 엔진 상태의 복제본이 아니라 상태 변화 이벤트를 전달하기 위한 구조이다.

6) Event Outbox 와 실시간 UI

(Demo 에서는 SSE Stream 을 구현했으나 실제 진행시에는 적용 여부 고민중)

Event Outbox 에 기록된 이벤트는 다양한 방식으로 소비될 수 있다.

- UI 상태 갱신
- 알림 서비스
- Webhook
- 이벤트 분석 시스템

기본적인 UI 상태 갱신은 polling 방식으로도 충분히 구현 가능하며,

필요한 경우 Event Outbox 를 기반으로 SSE(Server-Sent Events) 또는 WebSocket 을 통해 실시간 이벤트 스트리밍을 구현할 수 있다.

즉, Event Outbox 는 이벤트 기반 확장을 위한 선택적 아키텍처 요소이다.

7) 정리

PXM 은 프로세스 실행 이력을 다음 두 계층으로 관리한다.

1. Execution Log

엔진 내부 실행 기록을 저장하며 운영 모니터링 및 디버깅에 사용.

2. Event Outbox

외부 시스템 또는 UI 에서 소비할 수 있는 이벤트 스트림을 제공한다.

이 구조를 통해 PXM 은 실행 추적 기능과 이벤트 기반 확장 구조를 동시에 지원한다.

참고) 이와 같은 실행 이벤트 구조는 Camunda, Temporal, Zeebe 등 주요 Workflow 엔진에서도 유사하게 사용되는 일반적인 설계 방식.

9. Failure / Retry 모델

1) 개요

워크플로우 실행 과정에서 외부 시스템 호출 실패, 네트워크 오류, 일시적인 서비스 장애 등이 발생할 수 있다.

PXM 엔진은 이러한 실패 상황에서도 프로세스 실행이 안정적으로 유지되도록 **Retry 기반의 실패 처리 모델**을 사용한다.

Retry Model 은 다음 목표를 가진다.

- 일시적인 오류 자동 복구
- 외부 시스템 장애에 대한 내구성 확보
- 프로세스 실행 중단 최소화
- 안정적인 재시도 정책 제공

2) Failure 유형

워크플로우 실행 중 발생할 수 있는 오류는 다음과 같이 구분된다.

유형	설명
Transient Failure	일시적 오류 (네트워크, 외부 API 장애 등)
Permanent Failure	복구 불가능 오류 (비즈니스 규칙 실패 등)
Timeout	지정 시간 내 응답 없음

pxm 엔진은 이러한 오류를 기반으로 **재시도 가능 여부를 판단**한다.

3) Retry 정책

Service Node 실행 실패시 엔진은 정의된 Retry Policy 에 따라 재시도를 수행한다.
Retry 정책은 다음 요소로 구성된다.

항목	설명
max_attempts	최대 재시도 횟수
initial_delay	최초 재시도 지연 시간
backoff_strategy	지수 증가 방식
max_delay	최대 지연 시간 제한
retry_on	재시도 대상 오류

```
max_attempts = 5
initial_delay = 5s
backoff = exponential
max_delay = 5m
```

```
1 차 retry → 5 초
2 차 retry → 10 초
3 차 retry → 20 초
4 차 retry → 40 초
```

이 방식은 Exponential Backoff 전략으로,
외부 시스템 장애 시 과도한 호출을 방지하는데 효과적이다. (대부분 분산 시스템에서 표준)

4) Retry 와 Job Model

PXM 은 Job 기반 실행 모델을 사용하므로 Retry 는 **Job 재스케줄링 방식**으로 구현된다.

```
Service Node 실행
↓
실패 발생
↓
Retry 정책 확인
↓
engine_jobs 테이블에 retry job 생성
↓
run_at 시간 이후 워커 재실행
```

즉, Retry 는 별도의 로직이 아니라 **Job Queue 재등록** 방식으로 자연스럽게 처리된다.

5) Retry 상태 이벤트

Retry 가 발생하면 Event Outbox 에 이벤트가 기록된다.

Event	설명
RETRY_SCHEDULED	재시도 예약
RETRY_STARTED	재시도 실행
RETRY_EXHAUSTED	재시도 한도 초과

```
{
  "event_type": "RETRY_SCHEDULED",
  "instance_id": "inst_1",
  "node_id": "service_ad_apply",
  "payload": {
    "attempt": 2,
    "next_run_at": "2026-03-06T16:05:00Z"
  }
}
```

이를 통해 UI 및 운영자는 현재 재시도 상태를 확인할 수 있다.

6) 최종 실패 처리

재시도 횟수를 초과하면 Node 실행은 실패로 처리된다.
실행 결과는 다음 상태로 전이된다.

```
NODE_FAILED  
INSTANCE_FAILED
```

이 경우 운영자는 다음 대응을 선택할 수 있다.

- 수동 재실행
- 문제 수정 후 Resume
- 인스턴스 종료

7) 정리

Retry Model 을 통해 pxm 은 다음 특성을 제공한다.

- 외부 시스템 장애에 대한 자동 복구
- 안정적인 장기 실행 프로세스 지원
- 운영 개입 최소화
- 장애 상황에서도 프로세스 지속 가능

이 구조는 **Job 기반 워크플로우 엔진에서 일반적으로 사용되는 안정적인 실행 모델**이다.

참고) 유사한 Retry 기반 실행 모델은 Temporal, Camunda 등에서도 사용된다. 이들 시스템 또한 Job 기반 실행 모델과 Retry 정책을 결합하여 안정적인 워크플로우 실행을 제공한다.

PXM 은 Retry 만 별도로 다루지 않는다. Timer Node 대기, Approval Reminder, Escalation 과 같이 미래 시점에 실행되어야 하는 동작 역시 동일한 Job 재스케줄링 모델로 처리할 수 있다.

예를 들어 특정 시점 이후 재개가 필요한 경우 `engine_jobs` 에 `run_at` 을 가진 Job 을 생성하고, 워커는 해당 시점이 되었을 때 이를 실행한다.

10. Concurrency / Lock 모델

1) 개요

PXM 엔진은 Job 기반 실행 모델을 사용하며 여러 Worker가 동시에 Job을 처리할 수 있다.

이 환경에서는 ㄷ 동일한 Process Instance가 여러 Worker에 의해 동시에 실행되는 것을 방지해야 한다.

이를 위해 PXM 엔진은 다음 두 가지 메커니즘을 결합한 Concurrency Control 모델을 사용한다.

- Database Job Queue
- Instance Lock

이 구조를 통해 pxm은 분산 환경에서도 안전한 실행을 보장한다.

2) Job Queue 기반 실행

PXM은 Database 기반 Job Queue를 사용한다.

Worker는 다음 방식으로 Job을 가져온다.

```
SELECT *  
FROM engine_jobs  
WHERE run_at <= now()  
FOR UPDATE SKIP LOCKED  
LIMIT 1
```

이 방식의 특징

특징	설명
SKIP LOCKED	다른 Worker가 잡은 Job은 건너뛴다
Row Lock	동일 Job 중복 실행 방지
Natural Load Balancing	여러 Worker에 자동 분산

즉 Worker 여러 개가 동시에 실행되어도 같은 Job을 중복 실행하지 않는다.

3) Instance 동시 실행 문제

Job Queue 만으로는 다음 문제가 발생할 수 있다.

```
Worker A → instance_1 node 실행  
Worker B → instance_1 다른 node 실행
```

이 경우 동일 Instance 상태를 동시에 수정할 수 있어, Token 이동이나 Context 데이터가 충돌할 수 있다.

따라서 PXM 은 **Instance 단위 Lock** 을 사용한다.

4) Instance Lock

PXM 은 PostgreSQL Advisory Lock 을 사용하여 동일 Instance 의 동시 실행을 방지한다.

Worker 가 Instance 실행을 시작할 때 다음 Lock 을 획득한다.

```
pg_try_advisory_lock(hash(instance_id))
```

동작 방식

```
Worker A → Lock 획득 → 실행  
Worker B → Lock 실패 → Job 재시도
```

이를 통해 동일 Instance 는 동시에 하나의 Worker 만 실행할 수 있다.

5) Lease 기반 실행 보호

Worker 가 실행 중 장애로 종료되는 경우 Lock 이 장시간 유지될 수 있다.

이를 방지하기 위해 PXM 은 **Lease 기반 호보 구조**를 사용한다.

Instance 테이블에는 다음 필드가 존재한다.

필드	설명
lock_owner	실행 Worker
lock_until	Lock 만료 시간
heartbeat_at	마지막 heartbeat

실행 중 Worker 는 주기적으로 heartbeat 를 갱신한다.

```
heartbeat_at = now()  
lock_until = now() + lease_duration
```

Worker 장애 발생 시

```
lock_until < now()
```

조건을 만족하면 다른 Worker 가 Instance 실행을 다시 획득할 수 있다.

6) 실행 흐름

Instance 실행 과정은 다음과 같다.

```
Worker Job 획득
  ↓
Instance Lock 획득
  ↓
Node 실행
  ↓
Token 이동
  ↓
Instance 상태 업데이트
  ↓
Lock 해제
```

이 과정을 통해 PXM 은 분산 환경에서도 안전한 워크플로우 실행을 보장한다.

7) 정리

Concurrency / Lock Model 을 통해 pxm 은 다음 특성을 제공한다.

- Worker 수평 확장 가능
- Instance 동시 실행 방지
- 장애 상황 자동 복구
- 안정적인 장기 실행 워크플로우 지원

Job Queue 와 Instance Lock 을 결합함으로써 PXM 은 **높은 확장성과 실행 안정성을 확보한다.**

참고) 유사한 동시성 제어 구조는 다음 Temporal, Camunda 등에서도 사용된다. 이들 시스템 역시 Job 기반 실행과 Lock 메커니즘을 통해 분산 환경에서 워크플로우 실행을 관리한다.

11. Data 모델

1) 개요

PXM 엔진은 Database 를 **Source of Truth** 로 사용하는 실행 모델을 기반으로 한다.

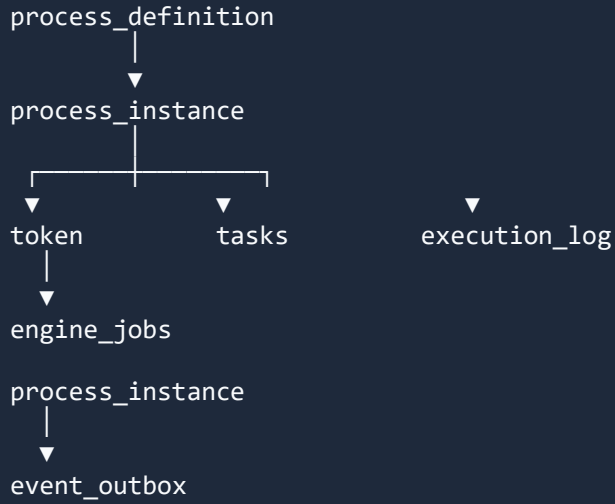
프로세스 실행 상태, 토큰 위치, 작업, 실행 로그 등 모든 Runtime 데이터는 데이터베이스에 저장되며 Worker 는 이를 기반으로 실행을 진행한다.

PXM Runtime 데이터 모델의 핵심 엔티티는 다음과 같다.

Entity	설명
process_definition	프로세스 정의
process_instance	프로세스 실행 인스턴스
token	현재 실행 위치
engine_jobs	실행 작업 큐
tasks	사용자 작업
execution_log	실행 기록
event_outbox	외부 이벤트 전달

2) 전체 구조

PXM Runtime 데이터 모델은 다음 구조를 가진다.



process_instance 는 워크플로우 실행의 기준 엔티티이며 토큰, 작업, 실행 로그, 실행 Job 등 모든 Runtime 데이터는 instance_id 를 기준으로 연결된다.

3) Process Definition

Process Definition 은 워크플로우 구조를 정의하는 템플릿이다.

필드	설명
id	프로세스 ID
version	프로세스 버전
nodes	노드 정의
edges	연결 정의
created_at	생성 시간

Process Definition 은 여러 Process Instance 를 생성할 수 있다.

4) Process Instance

Process Instance 는 프로세스 정의의 실제 실행 객체이다.

필드	설명
id	인스턴스 ID
process_definition_id	프로세스 정의
state	실행 상태
context	실행 데이터
created_at	생성 시간
updated_at	수정 시간

Instance 는 Runtime 데이터의 Root Entity 역할을 한다.

5) Token

Token 은 **Process Instance** 가 현재 어느 **Node** 에 있는지를 나타내는 실행 포인터다

필드	설명
id	토큰 ID
instance_id	인스턴스
node_id	현재 노드
status	토큰 상태

Parallel Gateway 에서는 하나의 Instance 에 여러 Token 이 존재할 수 있다.

6) Engine Jobs

Engine Jobs 는 Worker 가 실행할 작업 큐이다.

필드	설명
id	Job ID
instance_id	실행 인스턴스
token_id	실행 토큰
type	START / RESUME / RETRY / TIMER / REMINDER / ESCALATION
run_at	즉시 실행 또는 미래 예약 시점
attempt	재시도 횟수
status	queued / running / completed / failed
payload	재개 대상 task_id, escalation 대상 정보 등 Job 별 추가 데이터

Worker 는 Job Queue 에서 Job 을 가져와 실행한다.

Engine Jobs 는 Worker 가 실행할 통합 Job Queue 이다.

단순한 즉시 실행 작업뿐 아니라, START, RESUME, RETRY, TIMER, REMINDER, ESCALATION 과 같은 “지금 혹은 미래 시점에 다시 실행되어야 하는 동작”을 공통적으로 표현한다.

각 Job 은 type 과 run_at 을 통해 실행 의미와 예정 시점을 나타내며, 워커는 run_at <= now()인 Job 을 가져와 처리한다.

7) Tasks

Tasks 는 사용자 입력이 필요한 작업을 나타낸다.

예)

- 승인
- 검토
- 입력

필드	설명
id	Task ID
instance_id	인스턴스
node_id	노드
assignee	담당자
status	상태

승인/검토 Task 에 대한 Reminder 및 Escalation 은 Task 자체에 직접 스케줄 로직을 내장하기보다, 필요시 engine_jobs 에 후속 Job 을 생성하여 처리한다.

8) Execution Log

Execution Log 는 Node 실행 과정에서 발생한 내부 실행 기록이다.

필드	설명
id	Log ID
instance_id	인스턴스
token_id	토큰
node_id	노드
event_type	이벤트

Execution Log 는 실행 흐름 추적 및 디버깅에 사용된다.

9) Event Outbox

Event Outbox 는 외부 시스템에 전달할 이벤트를 저장한다.

필드	설명
id	Log ID
instance_id	인스턴스
token_id	토큰
node_id	노드
event_type	이벤트

Event Outbox 는 UI 갱신, 알림, 외부 시스템 연동 등에 사용된다.

10) 설계 의의

PXM Data Model 은 다음 원칙을 따른다.

- **Database Source of Truth**
- **Instance 중심 Runtime 모델**
- **Job 기반 실행 구조**
- **Event 기반 확장 지원**

이 구조를 통해 PXM 은 다음 특성을 제공한다.

- 안정적인 분산 실행
- 실행 상태의 일관된 관리
- 이벤트 기반 확장 가능성